Java SE 17 Programmer II（1Z0-826-JPN）　試験内容 8 ローカライズ　#17

ローカライズ

Locale・Locale.Category、ResourceBundle の探す順、NumberFormat・DecimalFormat・CompactNumberFormat、DateTimeFormatter・MessageFormat

各ページの下に、根拠にした資料（Oracle の試験内容チェックリスト、Java SE 17 API の Locale・ResourceBundle・ListResourceBundle・NumberFormat・DecimalFormat・CompactNumberFormat・MessageFormat・DateTimeFormatter・FormatStyle、The Java Tutorials の Internationalization）を書いた。コードの結果は JDK 17（Eclipse Temurin 17.0.21、既定のロケール en_US、ロケールのデータは CLDR）で実行して確かめた

全体像 ロケールに合わせて「文字」と「書式」を変える

Locale
言語＋国（ja_JP・en_US・de_DE）

↓ どれを使うか決める

ResourceBundle
画面の文言（キー → 値）を言語ごとに切りかえる

NumberFormat・DateTimeFormatter・MessageFormat
数値・通貨・日付・文の書式を切りかえる

値	en_US	de_DE	ja_JP
1234567.891	1,234,567.891	1.234.567,891	1,234,567.891
通貨	$1,234,567.89	1.234.567,89 €	￥1,234,568
2026-03-05 SHORT	3/5/26	05.03.26	2026/03/05
MMMM	March	März	3月

出典：API Locale・ResourceBundle・NumberFormat・DateTimeFormatter／JDK 17 での実行結果

第1章

Locale

作り方と、既定のロケール。

Locale の作り方

```
new Locale("fr")                // fr
new Locale("en", "GB")          // en_GB
new Locale("EN", "us")          // en_US（そろえられる）
new Locale("xx", "YY")          // xx_YY（例外にならない）
Locale.JAPAN                    // ja_JP
Locale.JAPANESE                 // ja
Locale.forLanguageTag("ja-JP")  // ja_JP
new Locale.Builder().setLanguage("de")
    .setRegion("CH").build()    // de_CH

new Locale();       // × no suitable constructor
Locale.of("en");    // × cannot find symbol（Java 19〜）
```

メソッド（en_GB）	結果
getLanguage()	en
getCountry()	GB
getDisplayCountry(Locale.ENGLISH)	United Kingdom
getDisplayLanguage(Locale.JAPAN)	英語
toString()	en_GB
toLanguageTag()	en-GB

出典：API Locale・Locale.Builder／JDK 17 でのコンパイルと実行の結果

既定のロケールと Category

```
Locale.setDefault(Locale.GERMANY);
Locale.getDefault()                          // de_DE
Locale.getDefault(Locale.Category.FORMAT)  // de_DE

Locale.setDefault(Locale.Category.FORMAT, Locale.FRANCE);
Locale.getDefault()                            // de_DE
Locale.getDefault(Locale.Category.FORMAT)   // fr_FR
Locale.getDefault(Locale.Category.DISPLAY)  // de_DE
```

Category	使われる場面
DISPLAY	言語・国などの名前の表示（getDisplayName など）
FORMAT	数値・通貨・日付の書式（NumberFormat.getInstance() など）

> setDefault(l) は両方を、setDefault(Category, l) はそのカテゴリだけを変える。JVM の中だけの変更。

出典：API Locale.setDefault・getDefault・Locale.Category／JDK 17 での実行結果

第2章

リソース・バンドル

どのファイルの、どのキーが使われるか。

探す順と親のつながり

ファイル	中身
Msg.properties	hello=Hello(base)、bye=Bye(base)、only=base-only
Msg_en.properties	hello=Hello(en)
Msg_fr.properties	hello=Bonjour(fr)、bye=Au revoir(fr)
Msg_fr_CA.properties	hello=Allo(fr_CA)

↓ 既定のロケールは en_US

求めた	見つかる	hello / bye / only
fr_CA	fr_CA	Allo / Au revoir / base-only
fr_FR	fr	Bonjour / Au revoir / base-only
ja_JP	en	Hello(en) / Bye(base) / base-only
Locale.ROOT	（基底）	Hello(base) / …

① バンドルを探す
求めた（国あり → 国なし）→ 既定のロケール（国あり → 国なし）→ 基底

② キーを探す
見つかったバンドル → 国を外した親 → 基底（既定のロケールのバンドルは親にならない）

ja_JP で Msg_ja がないと、基底ではなく既定のロケールの Msg_en が使われる。既定を fr_FR にすると Msg_fr。

getNoFallbackControl を渡すと、既定のロケールを飛ばして基底へ。

出典：API ResourceBundle.getBundle・ResourceBundle.Control／JDK 17 での実行結果

クラスのバンドルと例外

```
public class Msg_de extends ListResourceBundle {
  protected Object[][] getContents() {
    return new Object[][] {
      {"hello", "Hallo(de.class)"}, {"num", 42} };
  }
}
// Msg_de.properties もあるとき
ResourceBundle de = ResourceBundle.getBundle("Msg", Locale.GERMANY);
de.getString("hello")  // Hallo(de.class)（クラスが先）
de.getObject("num")    // 42
de.getString("num")    // ClassCastException
de.getString("zzz")    // MissingResourceException
```

場面	結果
キーがない	MissingResourceException（非チェック）
バンドルがない	MissingResourceException
値が String でない	getString で ClassCastException
keySet()	親のキーも含む（fr_CA なら bye・hello・only）
Properties.getProperty	なければ null（既定値を渡せばそれ）

出典：API ResourceBundle・ListResourceBundle・Properties／JDK 17 での実行結果

第3章

数値のフォーマット

NumberFormat と DecimalFormat。

NumberFormat の種類

メソッド（Locale.US）	例
getInstance	1234567.891 → 1,234,567.891
getCurrencyInstance	1234567.891 → $1,234,567.89
getPercentInstance	0.256 → 26%、1.5 → 150%
getIntegerInstance	2.5 → 2、3.5 → 4（HALF_EVEN）
getCompactNumberInstance(SHORT)	1234 → 1K、1500000 → 2M、999 → 999
getCompactNumberInstance(LONG)	1500000 → 2 million

> NumberFormat は抽象クラス（new できない）。丸めの既定は HALF_EVEN。

```
NumberFormat nf = NumberFormat.getInstance(Locale.US);
nf.parse("1,234.5abc")  // 1234.5（読めるところまで）
nf.parse("12x3")        // 12
nf.parse("3.0")         // Long の 3
nf.parse("abc")         // ParseException（チェック例外）
NumberFormat.getInstance(Locale.GERMANY)
    .parse("1.234,5")   // 1234.5

c.setMaximumFractionDigits(1);
c.format(1234)          // 1.2K
```

出典：API NumberFormat・CompactNumberFormat・ParseException／JDK 17 での実行結果

DecimalFormat のパターン

記号	意味
0	数字（なければ 0 を出す）
#	数字（なければ出さない）
.	小数点
,	けた区切り
%	100 倍してパーセント
'文字'	そのまま出す

```
new DecimalFormat("#,##0.00").format(1234.5)  // 1,234.50
new DecimalFormat("000.#").format(5.25)       // 005.2
new DecimalFormat("#.##").format(0.5)         // 0.5
new DecimalFormat("0.0").format(2.25)         // 2.2 (HALF_EVEN)
new DecimalFormat("$#,###").format(1234567)   // $1,234,567

String.format(Locale.GERMANY, "%,.2f", 1234567.891)
// 1.234.567,89
```

> ちょうど半分は偶数のほうへ（5.25 → 5.2、2.5 → 2、3.5 → 4）。

出典：API DecimalFormat・RoundingMode.HALF_EVEN・String.format／JDK 17 での実行結果

第4章

日付・時刻とメッセージ

DateTimeFormatter と MessageFormat。

DateTimeFormatter

```
LocalDateTime dt = LocalDateTime.of(2026,3,5,14,7,9);
"yyyy/MM/dd"  // 2026/03/05
"y-M-d"       // 2026-3-5
"yyyy/mm/dd"  // 2026/07/05（mm は分！）
"hh:mm a"     // 02:07 PM (US)
"HH:mm:ss"    // 14:07:09
"MMM MMMM EEE EEEE"  // Mar March Thu Thursday
"'Today is' yyyy 'at' H'h'"  // Today is 2026 at 14h

LocalDate.of(2026,3,5).format(ofPattern("HH:mm"))
// UnsupportedTemporalTypeException:
//   Unsupported field: HourOfDay
ofPattern("yyyy-MM-dd q b")
// IllegalArgumentException: Unknown pattern letter: b
```

FormatStyle	US	JAPAN
SHORT	3/5/26	2026/03/05
MEDIUM	Mar 5, 2026	2026/03/05
LONG	March 5, 2026	2026年3月5日
FULL	Thursday, March 5, 2026	2026年3月5日木曜日

> ofLocalizedDateTime の LONG・FULL はタイムゾーンが要る。LocalDateTime では DateTimeException（Unable to extract ZoneId）。

出典：API DateTimeFormatter・FormatStyle・UnsupportedTemporalTypeException／JDK 17 での実行結果

MessageFormat

```
MessageFormat.format("{0} has {1} items, {0}!", "Ann", 1234)
// Ann has 1,234 items, Ann!

MessageFormat.format("{1} {0} {2}", "a", "b")
// b a {2}（値がなければそのまま）

MessageFormat.format("It''s {0} '{1}'", "x", "y")
// It's x {1}

new MessageFormat("{0,number,#.#} {1,number,percent}",
    Locale.US).format(new Object[]{3.14159, 0.5})
// 3.1 50%
```

{番号}
何度でも使える。数値はロケールで区切られる

' と ''
'…' の中はそのまま、'' で ' を 1 つ

リソース・バンドルと組み合わせる
パターンをバンドルに入れ、値を MessageFormat で入れる

出典：API MessageFormat／JDK 17 での実行結果

本番での問われ方

ねらい	見るところ
Locale	toString は _、toLanguageTag は -、new Locale() はない、Category
バンドルを探す順	国を外す → 既定のロケール → 基底、クラスが先
キーを探す範囲	親（国・言語を外したもの）と基底だけ、なければ MissingResourceException
数値	ロケールで区切りが変わる、HALF_EVEN、parse は読めるところまで
日付	MM と mm、HH と hh、LocalDate に時刻は UnsupportedTemporalTypeException
MessageFormat	{番号}、' の扱い、値のない番号は残る

出典：試験内容チェックリスト 8／本資料の問題演習（全 30 問、JDK 17 で確認）